

NAME:

Practice Problems for Week 2

Please complete, bring to class (written answers) and submit on gradescope (code) by end of day on **Monday 4/13/26**.

You will be submitting the following files:

- square_chain.py
- retirement.py
- All the programming files **MUST** be named as instructed

Programming Part 1: Turtle Stairs

1. Create a file called square_chain.py for your code.
2. Write a Python program using turtle to draw something that draws a diagonal chain of 10 squares:

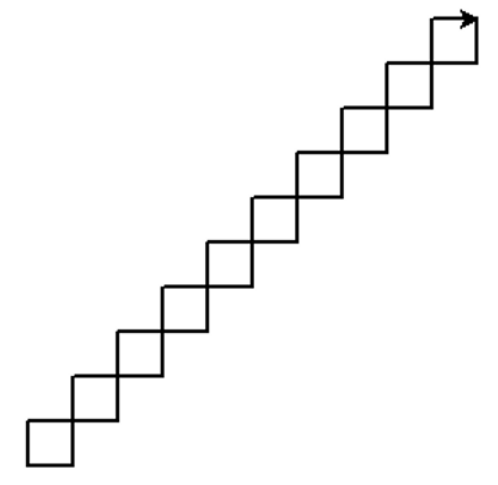


Figure 1: Turtle Stairs

There are many different ways (algorithms) that you can draw this pattern. Some may (or may not) leave the turtle in the same position and orientation that you see here. It does not matter if the turtle has the same position and orientation as what you see here.

3. Add comments to explain how your program works.
 - Header comment with Name, Email, Date, Assignment, Description
 - In-line comments describing the algorithm guiding your program
 - Docstrings for any functions, if you define new functions

Programming Part 2: Retirement Calculator

It's never too early to start saving for retirement. For this assignment you're going to write a simple retirement calculator.

- A run of the program may look like the following (user input in bold):

```
How much do you plan to save each month? (in dollars) **100**
How old are you? (in years) **20**
At what age do you plan to retire? (in years) **40**
What yearly rate of return do you expect? (in percentage points) **5**
Your savings at age 21 are: $ 1200
Your savings at age 22 are: $ 2460
Your savings at age 23 are: $ 3783
Your savings at age 24 are: $ 5172
Your savings at age 25 are: $ 6630
Your savings at age 26 are: $ 8162
Your savings at age 27 are: $ 9770
Your savings at age 28 are: $ 11458
Your savings at age 29 are: $ 13231
Your savings at age 30 are: $ 15093
Your savings at age 31 are: $ 17048
Your savings at age 32 are: $ 19100
Your savings at age 33 are: $ 21255
Your savings at age 34 are: $ 23518
Your savings at age 35 are: $ 25894
Your savings at age 36 are: $ 28388
Your savings at age 37 are: $ 31008
Your savings at age 38 are: $ 33758
Your savings at age 39 are: $ 36646
Your savings at age 40 are: $ 39679
```

1. Start by creating a file called retirement.py for your code.
2. Next, start writing code to get input from the user and store the information in variables.
 - The program should start by asking the user for the following information:
 - The amount they plan to save each month
 - Their starting age
 - Their planned retirement age
 - The yearly rate of return they expect (i.e. growth through interest or dividends).
3. Now design an algorithm to compute the amount that accumulates each year. Write out the algorithm in a human language (e.g. English) before writing it in code. Include this as your description in your header comment.
 - The program should show how the user's savings grow over the course of their careers. For example, if a 20 year-old starts saving \$100 a month, retires at 40, and gets a 5% rate or return, then they should have about \$39,679 by the time they retire.

- To simplify things, we assume that interest is added once per year before the user has contributed any of their monthly savings.
 - This means that, for the first year, the rate of return is zero, so they only get what they saved over the course of the year (i.e. \$1200).
 - For subsequent years, calculate the interest gained, then add the yearly contribution. So for year 2:
 - * the interest will be $\$1200 \times 5\% = \60
 - * the savings at the end of year will be $\$1200 + \$60 + \$1200 = \2460 .
4. Implement the program in Python
 5. Test the program with different values to ensure that it works correctly.
 6. If you haven't already, add comments explaining how the program works.

Written Questions**1. Consider the following (incomplete) Python program:**

```
x = int(input("Please input a positive integer: "))
counter = x
for step in range(0, 10):
```

Complete the missing lines of code above so that the program will produce the following output if the user inputs 3.

```
3
6
9
12
15
18
21
24
27
30
```

- **Notes:** You do not need to introduce any additional variables. There is more than one way to achieve this result.

2. Consider the following (incomplete) Python program to list the first few Fibonacci numbers:

The Fibonacci numbers are the following sequence of numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

- The first two numbers are 0 and 1.
- After that, each number can be calculated by adding the previous two together. For example:
 - the third number is 1 (0+1)
 - the fourth number is 2 (1+1)
 - the fifth number is 3 (2+1), etc.

They are named Fibonacci numbers after the Italian mathematician who first described them in Europe. Patterns based on the Fibonacci sequence show up in surprisingly many contexts in math, biology, and art. (See the Wikipedia entry if you want to know more.)

Below is the (incomplete) Python program to list the first few Fibonacci numbers.

The user gets to determine how many Fibonacci numbers to list.

```
1. print ("How many Fibonacci numbers would you like to see?")
2. limit = int (input("Please input a number greater than 2: "))
3.
4. fib_0 = 0
5. fib_1 = 1
6.
7. print(fib_0)
8. print(fib_1)
9. for count in range(      ):
10.     next_fib =
11.     print(next_fib)
```

- For questions a-d, assume that the program has been completed and functions correctly.

2.a. Draw a memory diagram that shows the- associations after line 5 has been executed. Assume that the user inputted 10. Also show what will have been printed to the screen at that point.

2.b. Draw a memory diagram that shows the - associations that should exist after line 11 has been executed for the first time. Also show what line 11 will print to the screen when it is executed for the first time.

2.c. What should the - associations be when line 11 has been executed for the second time? Draw a memory diagram.

2.d. Draw a memory diagram that shows the - associations after the last number has been printed (i.e. at the end of the program).

2.e. Complete the code above so that the program prints the first n Fibonacci numbers, where n is determined by the user. You may need to add new lines of code.

3. Consider two Python programs:

- Both programs on the next page will help Reeborg harvest the carrots in the Harvest 1 world:

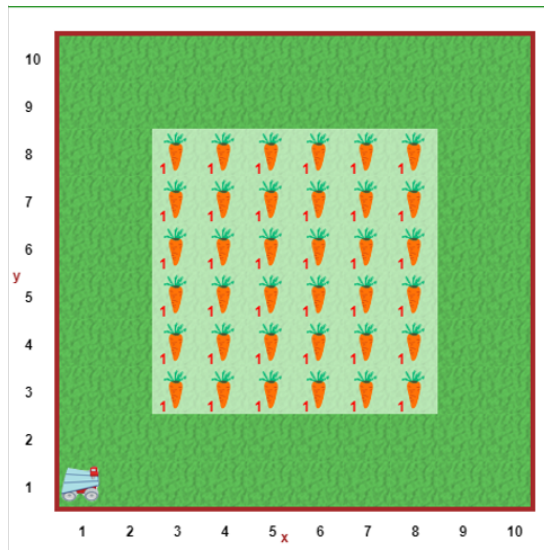


Figure 2: Reeborg Carrots

- Both programs define a number of functions. Unfortunately, the program designers were from the bad old days when names needed to be as short as possible, so their function names are essentially meaningless. Come up with better names for each of the functions that they defined.

-
-
-
-
-
-
-
-

Program 1

```
1 def turn_right():
2     turn_left()
3     turn_left()
4     turn_left()
5
6 def a():
7     move()
8     turn_left()
9     move()
10    move()
11    turn_right()
12    move()
13
14 def b():
15     for carrot in range(0, 6):
16         take()
17         move()
18
19 def c():
20     turn_left()
21     move()
22     turn_left()
23     move()
24
25 def d():
26     turn_right()
27     move()
28     turn_right()
29     move()
30
31 a()
32 for double_row in range(0, 3):
33     b()
34     c()
35     b()
36     d()
```

Program 2

```
1 def turn_right():
2     turn_left()
3     turn_left()
4     turn_left()
5
6 def e():
7     move()
8     turn_left()
9     move()
10    move()
11    turn_right()
12    move()
13    take()
14
15 def f():
16     for carrot in range(0, 5):
17         move()
18         take()
19
20 def g():
21     f()
22     turn_left()
23     move()
24     turn_left()
25     take()
26
27 def h():
28     f()
29     turn_right()
30     move()
31     turn_right()
32     take()
33
34 e()
35 for double_row in range(0, 2):
36     g()
37     h()
38     g()
39     f()
```

3.i Now that you've come up with better names for the functions, which of the solutions do you think is better and why?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

4. Check-in

4.a. Were you able to complete all of the programming activities (yes/no)?

.....

4.b. What, if anything, did you struggle with when working on the activities?

.....

.....

.....

.....

4.c. If you managed to overcome the challenges, how did you do it?

.....

.....

.....

.....